

milosvasic-ru

Miloš Vasić

Héros

Ingénieur AI spécialisé dans la construction de systèmes de développement AI vérifiables.

Je construis la partie de l'ingénierie AI qui distingue un produit fiable d'une démonstration impressionnante : l'infrastructure LLM multi-fournisseurs capable de survivre à la défaillance d'un prestataire, les agents autonomes et l'orchestration qui maintiennent le travail sur les rails, ainsi que les couches de gouvernance et d'assurance qualité qui empêchent un système AI de mentir discrètement sur ses capacités. Transformer un grand modèle de langage en un produit réellement livrable relève avant tout d'un problème de discipline, et c'est précisément cette discipline qui fait ma spécialité. Ma boussole repose sur une règle unique : une fonctionnalité n'est terminée que lorsqu'un utilisateur réel peut s'en servir et qu'il existe des preuves tangibles pour le démontrer.

Résumé

Je travaille principalement dans le domaine **Go**, avec **Kotlin / Kotlin Multiplatform, TypeScript/React, Python, Swift** et **Shell** selon les besoins. Ce qui m'importe le plus, c'est la manière dont le travail est *structuré* : non pas un amas d'applications ponctuelles, mais une flotte — de grandes applications produits reposant sur des dizaines de modules petits, découplés et testés indépendamment, tous héritant d'une **Constitution** d'ingénierie partagée sous forme de sous-module Git. Ce choix architectural unique est ce qui permet à l'ensemble du travail de s'enrichir mutuellement : les corrections et améliorations se propagent instantanément à tous les projets, les nouveaux produits s'assemblent à partir de composants déjà éprouvés, et chaque fonctionnalité annoncée s'appuie sur un test produisant des preuves plutôt que sur une simple affirmation. Il s'agit d'une

ingénierie conçue pour inspirer confiance à grande échelle, même lorsqu'elle est menée par une seule personne. Cette page part de cette vue d'ensemble pour descendre jusqu'aux projets individuels ; chacun renvoie vers sa page produit complète.

Ma méthode de travail — gouvernance et assurance qualité avant tout

Avant les produits, la discipline qui les sous-tend :

- **HelixConstitution** — Je maintiens un recueil universel et héritable de règles d'ingénierie, distribué sous forme de sous-module Git à travers une flotte de plus de 140 dépôts. Il codifie des garde-fous anti-bidonnage, une immunité aux faux positifs, la sécurité des données et des hôtes, ainsi que des règles de couverture ; les projets peuvent les renforcer, mais jamais les affaiblir, et chaque garde-fou est couplé à un test de mutation prouvant son efficacité. → voir la page produit HelixConstitution.
- **HelixQA** — Je développe une orchestration d'assurance qualité anti-bidonnage exécutant des batteries de tests écrits et des sessions QA entièrement autonomes, pilotées par LLM et la vision, sur Android, Android TV, Web et Desktop, ne validant un PASS qu'après avoir capturé des preuves d'exécution. → voir la page produit HelixQA.

Mes réalisations au sein de la famille Helix

La gamme Helix couvre l'ensemble du cycle de développement AI. Par ordre de priorité :

- **HelixTrack** — une alternative JIRA du monde libre ; le fleuron de la gamme Helix-Track.
- **HelixAgent** — un service LLM en ensemble avec débat multi-tours entre modèles et sélection de fournisseurs basée sur la vérification.
- **HelixCode** — une plateforme de développement AI distribuée répartissant le travail entre des travailleurs gérés par SSH avec points de contrôle et retour arrière automatiques.
- **HelixLLM** — un binaire unique avec six modes servant des API compatibles OpenAI et Anthropic via HTTP/3, avec inférence locale llama.cpp et une chaîne de repli notée.
- **HelixCluster** — un système d'exploitation distribué pour le calcul AI, des GPU de datacenter aux appareils mobiles en périphérie.
- **LLMProvider** — une interface unique pour 43 fournisseurs intégrant disjoncteurs, nouvelles tentatives et état de santé.

- **LLMOrchestrator** — un plan de contrôle unique pour tous les agents de codage CLI sans interface.
- **LLMsVerifier** — vérifier, surveiller, optimiser : la source unique de vérité pour les métadonnées LLM/fournisseur/vérification.
- **HelixMemory, HelixSkills, HelixSpecifier, HelixBuilder, HelixTranslate, HelixTerminator, HelixGitpx, HelixOTA, HelixPlay** — mémoire d'agent, compétences d'agent gouvernées, développement piloté par spécifications, construction d'applications AI, traduction de livres vérifiée, terminaux à confiance zéro, Git fédéré, mises à jour OTA sans brique, et cloud gaming auto-hébergé.

Contenu

Mes réalisations dans les outils vasic-digital

Outils de niveau produit que j'ai développés et que je maintiens (chacun dispose d'une page produit complète) :

- **Catalogizer** — gestion multi-protocoles (SMB/FTP/NFS/WebDAV/local) de collections médias chiffrées et auto-hébergées, avec une interface utilisateur Go/Gin API et React.
- **Courses-Creator** — pipeline de conversion Markdown vers vidéo intégrant un enrichissement multi-LLM, des fonctionnalités TTS et des lecteurs pour desktop, mobile et web.
- **VisionEngine** — boîte à outils Go découplée combinant vision par ordinateur classique et vision multi-fournisseurs LLM pour l'analyse d'interfaces et la génération de graphes de navigation.
- **DocProcessor** — transforme la documentation en une cartographie vérifiable des fonctionnalités pour l'automatisation des tests QA (extraction LLM ou heuristique).
- **Docs Chain** — moteur de synchronisation bidirectionnelle et atomique de documents/bases de données avec hachage de contenu.
- **Herald** — notifications multi-canaux fiables avec résolution d'intention en trois niveaux et langage naturel.
- **task_bridge** — moteur de synchronisation découplé et bidirectionnel pour tâches/tableaux (structure P1 ; logique de synchronisation en cours).
- **Suite de modules réutilisables Vasic Digital** — la « bibliothèque standard » `digital.vasic.*` composée de modules d'infrastructure, de primitives AI et de garde-fous.

Héritage infrastructurel (Server Factory)

Avant la gamme AI, ma chaîne d'outils DevOps : **Mail Server Factory** (JSON déclaratif → serveurs mail Dockerisés entièrement provisionnés, avec 439 tests validés et une porte SonarQube propre), le **Cadre de base Server Factory** sur lequel il s'appuie, ainsi que des outils de gestion d'images VM (**Qemu-Utills**, **Parallels-Utills**) et des usines de services associées.

En une phrase

Je ne livre pas des coches vertes — je livre des systèmes AI accompagnés des preuves qu'ils fonctionnent réellement, et des mécanismes de gouvernance qui les maintiennent ainsi.

Contact

Ouvert aux postes seniors en ingénierie AI/plateforme à l'international.

- **Email** : milos85vasic@gmail.com · i@mvasic.ru
- **GitHub** : [milos85vasic](https://github.com/milos85vasic)
- **Telegram** : [@milos85vasic](https://t.me/milos85vasic)